

BACKEND SYSTEM DOCUMENTATION

P-FUND RESTful APIs

A detailed directory structure and codebase guide for the P-FUND project funding and management platform.

This backend is powered by Laravel, leveraging Sanctum token-based authentication and a robust role-based access control system.

Author: Laravel Development Team

Target Environment: Local/Staging/Production

Date Generated: May 25, 2026

API Version: 1.0 (v1)

1. System Overview & Architecture

P-FUND is a secure API-driven portal built to coordinate the funding, management, and assessment of academic and developmental projects. The backend is structured using the Model-View-Controller (MVC) paradigm provided by Laravel, though it functions strictly as a headless API provider. All actions returned map onto one of four core roles: Creator, Admin, Vetter, or Sponsor.

Core Security & Routing Flow

- 1. Authentication: Performed via Laravel Sanctum. Requests require an 'Authorization: Bearer <token>' header.**
- 2. Role Enforcement: Middleware intercepts routes to ensure only appropriate user roles access matching domains (e.g. Creator, Admin, Vetter, Sponsor).**
- 3. Structured Responses: The application uses a unified response trait to output a standard JSON format consisting of success status, user-facing message, and payload details.**

2. API Route Configuration

The routing layer maps URIs to specific controller logic. All routes are prefixed with '/api/' as configured by Laravel.

routes/api.php

The master routing file. It houses all endpoint declarations, separating them into:

- Public Auth (register, login, forgot password, reset password)
- Authenticated Auth (logout, email verification, resend OTP)
- Shared Protected (profile details, global chat index, user notifications, and file downloads)
- Role-Specific Groups (creator, admin, vetter, sponsor routes)

routes/api/v1/ (admin.php, auth.php, projects.php, sponsor.php, vetter.php)

Modular placeholder routing files set up to split API routing logic when scaling out.

routes/web.php & console.php

Contains the web endpoint stub and registers custom Artisan console commands.

3. HTTP Controllers (app/Http/Controllers/Api/V1/)

Authentication & User Profile Controllers

AuthController.php

Handles user signup (collecting details like education level, phone, nationality, and specialties), email verification via a 6-digit OTP code, portal-aligned logins (validates if a Creator is logging into the Creator app, or a Sponsor into the Sponsor app, etc.), account suspension checks, password resets, and session logouts.

ProfileController.php

Provides endpoints for users to retrieve their own profiles, update contact/personal details, change their passwords, and upload profile avatar images.

Creator Interface Controllers

ProjectController.php

Enables Creators to list their projects, create a new project application (budget, overview, milestones), edit drafts, delete draft projects, submit projects for official review, and upload/delete proposal documents.

DeliverableController.php

Allows Creators to submit evidence or files for milestones and trace deliverables as they go through review.

Admin Management Controllers

AdminProjectController.php

Provides the admin dashboard with project listings, project review controls (approve, reject, request changes), and milestone timeline customization.

AdminUserController.php

Allows administrators to provision new users, change user roles, and suspend/unsuspend user accounts to block login access.

ActivityLogController.php

Serves database audit trail logs, allowing admins to oversee system operations, signups, and reviews.

4. HTTP Controllers (Continued)

Vetting & Review Controllers

VetterProjectController.php

Used by scientific or domain Vettors. Provides a project queue, overall stats, ability to request additional updates from creators, reject proposal plans, and mark specific project milestones as complete.

VetterDeliverableController.php

Enables Vettors to inspect submitted deliverables for milestones and assign status (approved or needs correction).

Sponsor Interface Controllers

SponsorProjectController.php

Houses endpoints for funding parties. Provides dashboard stats (total committed funds, active projects), lists of approved projects open for funding, project interaction tracking, and listing the sponsor's funded project portfolio.

Shared & Utility Controllers

ChatController.php & ChatMessageController.php

Controls fetching and storing message streams for the global in-app communication channel.

NotificationController.php

Fetches notification items for the logged-in user, showing alerts on project updates or reviews, and handles marking items as read.

DocumentController.php

Handles file streaming and secure downloads for proposal documents and milestone deliverables.

5. Models & Schema Structure (app/Models/)

Database Relationships & Models

User.php

Defines user attributes, hashes passwords, enforces active/inactive states, and maps relationships to projects and activity logs. Uses Laravel Sanctum's HasApiTokens.

Project.php

Represents the funding proposals. Linked to a Creator (User), documents, milestones, commitments, and tracks application status (draft, submitted, approved, rejected, updates_required).

Milestone.php

Represents individual targets or funding stages of a project, tracking completion progress.

ProjectDeliverable.php

Saves submission entries, file paths, and feedback for milestone deliverables.

ProjectDocument.php

Tracks the physical file uploads, document names, and paths associated with project applications.

SponsorCommitment.php

Tracks the funding records connecting a Sponsor (User) and a Project.

SponsorProjectInteraction.php

Records clicks, bookmarks, or interest clicks by sponsors on active campaigns.

EmailVerificationCode.php

Stores temporary 6-digit OTP codes, associated user IDs, and helper methods to determine expiration.

ActivityLog.php, Notification.php, ChatMessage.php, Message.php

Models supporting database logging, notification delivery, and global/private messaging.

6. Middleware, Migrations & Environment Config

System Middleware (app/Http/Middleware/)

EnsureRole.php

Custom middleware validating the current user's role against route requirements (e.g. aborts with 403 Forbidden if a Sponsor tries to access Creator endpoints).

ForceJsonResponse.php

Ensures all uncaught exceptions, validations, or system responses are formatted as 'application/json' instead of standard HTML redirects.

Database Migrations (database/migrations/)

21 Schema Migration Files

Step-by-step definition of the SQLite database tables. Tracks user details, security tokens, projects, milestones, deliverables, logs, and notifications. Runs sequentially using standard Laravel migrations.

System Configurations (Root Directory)

.env & .env.example

Environment configuration containing environment variables (database path, app key, mail server host and credentials, app URL).

composer.json & composer.lock

PHP package manager configuration describing the project's dependencies and locking them to specific versions.

artisan

The command-line tool command script to perform routine tasks like starting a dev server, running migrations, or seeders.